

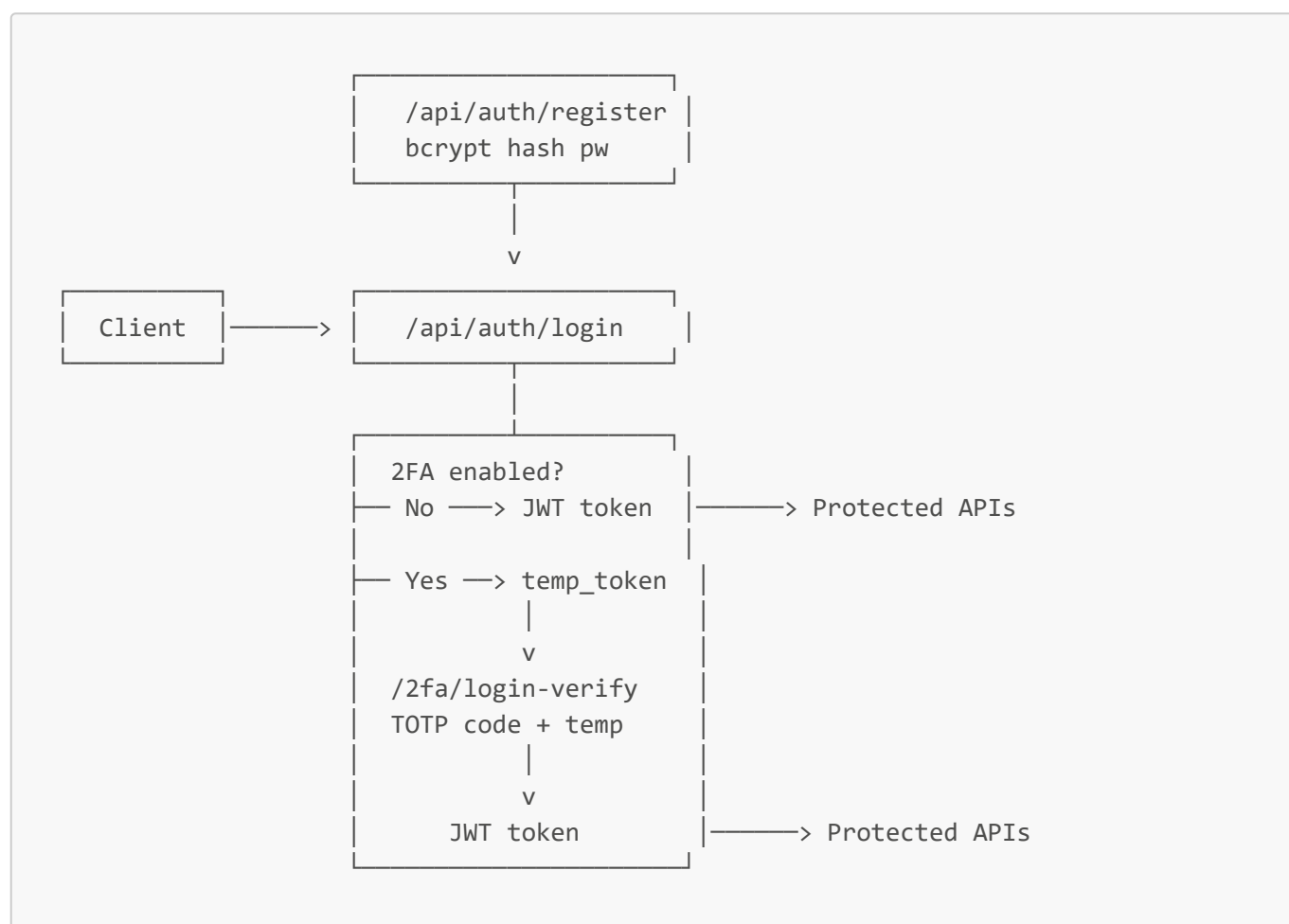
# Auth & Security — Xác thực và Bảo mật

Tài liệu giải thích hệ thống xác thực người dùng, bao gồm JWT token, mã hóa mật khẩu bcrypt, và xác thực hai yếu tố (2FA) bằng TOTP.

## Tham chiếu mã nguồn:

- `src/auth/router.py` — API endpoints
- `src/auth/utils.py` — JWT, bcrypt utilities
- `src/database/models.py` — User, ChatSession, Message models

## 1. Tổng quan luồng xác thực



## 2. Mã hóa mật khẩu — bcrypt

Tại sao bcrypt?

Mật khẩu **không bao giờ** được lưu dạng plain text. bcrypt là thuật toán hash chuyên dụng cho mật khẩu với các đặc tính:

- **Salt tự động:** Mỗi lần hash tạo salt ngẫu nhiên, cùng mật khẩu cho hash khác nhau
- **Cost factor:** Cấu hình được độ chậm (rounds), chống brute-force

- **Chống Rainbow Table:** Salt ngẫu nhiên làm bảng hash dựng sẵn vô hiệu

## Triển khai

```
from passlib.context import CryptContext
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# Hash khi đăng ký
hashed = pwd_context.hash("password123")
# → "$2b$12$LJ3m4iy1..." (60 ký tự, chứa salt + hash)

# Verify khi đăng nhập
pwd_context.verify("password123", hashed) # → True
```

Thư viện `passlib` tự quản lý salt, rounds (mặc định 12 rounds =  $2^{12}$  iterations).

---

## 3. JWT Token — JSON Web Token

### Cấu trúc JWT

```
Header.Payload.Signature

Header: {"alg": "HS256", "typ": "JWT"}
Payload: {"sub": "user@email.com", "user_id": 1, "exp": 1234567890}
Signature: HMAC-SHA256(header + payload, SECRET_KEY)
```

## Triển khai

```
from jose import jwt

token = jwt.encode(
    {"sub": email, "user_id": user_id, "exp": expire_time},
    SECRET_KEY,
    algorithm="HS256"
)
```

### Cấu hình

| Tham số        | Giá trị             | Nguồn                                                          |
|----------------|---------------------|----------------------------------------------------------------|
| Algorithm      | HS256 (HMAC-SHA256) | <code>config.py</code>                                         |
| SECRET_KEY     | Chuỗi ngẫu nhiên    | <code>.env</code>                                              |
| Thời gian sống | 7 ngày (mặc định)   | <code>.env</code> ( <code>ACCESS_TOKEN_EXPIRE_MINUTES</code> ) |

## Bảo vệ API endpoints

Mọi endpoint cần xác thực đều dùng `Depends(get_current_user)`:

```
@router.get("/me")
def get_me(current_user: User = Depends(get_current_user)):
    return current_user
```

`get_current_user` tự động:

1. Trích xuất token từ header `Authorization: Bearer <token>`
2. Decode và verify signature
3. Kiểm tra expiration
4. Query user từ database theo email trong payload
5. Raise 401 nếu bất kỳ bước nào thất bại

---

## 4. Xác thực hai yếu tố (2FA) — TOTP

TOTP là gì?

**TOTP (Time-based One-Time Password)** — mã xác thực 6 chữ số thay đổi mỗi 30 giây, dựa trên:

- **Secret key:** chuỗi base32 được chia sẻ giữa server và app authenticator
- **Thời gian hiện tại:** chia thành các window 30 giây

Công thức:  $TOTP = \text{HMAC-SHA1}(\text{secret}, \text{floor}(\text{time} / 30)) \bmod 10^6$

Luồng thiết lập 2FA

1. POST /2fa/setup
  - Server tạo secret ngẫu nhiên (`pyotp.random_base32()`)
  - Lưu secret vào `user.totp_secret`
  - Tạo QR code (SVG → base64) từ otpauth URI
  - Trả về `{secret, qr_code}`
2. User quét QR bằng Google Authenticator / Authy
3. POST /2fa/verify (body: `{totp_code: "123456"}`)
  - Server verify mã TOTP (`valid_window=1`: chấp nhận +-30s)
  - Nếu đúng: `is_2fa_enabled = True`
  - Nếu sai: HTTP 400

Luồng đăng nhập khi 2FA bật

1. POST /login (email + password)
  - Verify password thành công

- Phát hiện 2FA enabled
- Tạo temp\_token (JWT, purpose="2fa\_verify", sống 5 phút)
- Trả về {requires\_2fa: true, temp\_token: "..."}
  2. POST /2fa/login-verify (body: {temp\_token, totp\_code})
    - Decode temp\_token, kiểm tra purpose="2fa\_verify"
    - Verify TOTP code
    - Nếu đúng: trả JWT thật (sống 7 ngày)

### Tại sao dùng temp\_token?

temp\_token đóng vai trò **proof of password** — chứng minh người dùng đã pass bước 1 (password). Nếu không có temp\_token, bước 2 sẽ không biết ai đang xác thực. temp\_token sống ngắn (5 phút) để giảm rủi ro bị đánh cắp.

## 5. Database Schema

```

Users
├── id                INT, PK, auto-increment
├── full_name         VARCHAR(255), NOT NULL
├── email             VARCHAR(255), UNIQUE, INDEX, NOT NULL
├── hashed_password   VARCHAR(255), NOT NULL
├── totp_secret       VARCHAR(64), NULLABLE
├── is_2fa_enabled    BOOLEAN, DEFAULT FALSE (server_default='0')
└── created_at       DATETIME, DEFAULT utcnow()

ChatSessions
├── id                VARCHAR(50), PK, INDEX          -- UUID sinh từ frontend
├── user_id           INT, FK → users.id
├── title             VARCHAR(255)                    -- 50 ký tự đầu của message, thêm
"..."
└── updated_at       DATETIME, DEFAULT utcnow(), ON UPDATE utcnow()

Messages
├── id                INT, PK, auto-increment
├── session_id        VARCHAR(50), FK → chat_sessions.id
├── role              VARCHAR(50), NOT NULL           -- "user" | "assistant"
├── content           TEXT, NOT NULL
├── sources           TEXT, NULLABLE                  -- JSON string của danh sách doc
├── rewritten_query   VARCHAR(255), NULLABLE          -- Câu truy vấn sau khi LLM
rewrite
└── created_at       DATETIME, DEFAULT utcnow()
  
```

Định nghĩa chính xác tại `src/database/models.py` — SQLAlchemy tự sinh DDL qua `Base.metadata.create_all(bind=engine)` trong `api/main.py`.

Quan hệ: **User 1:N ChatSession 1:N Message** Cascade: Xóa user → xóa tất cả session → xóa tất cả message.

## 6. Danh sách API Endpoints

Auth (/api/auth)

| Method | Endpoint          | Mô tả                    | Auth?      |
|--------|-------------------|--------------------------|------------|
| POST   | /register         | Đăng ký tài khoản        | Không      |
| POST   | /login            | Đăng nhập                | Không      |
| GET    | /me               | Lấy thông tin user       | JWT        |
| PUT    | /update-profile   | Cập nhật họ tên          | JWT        |
| POST   | /change-password  | Đổi mật khẩu             | JWT        |
| GET    | /2fa/status       | Kiểm tra 2FA bật/tắt     | JWT        |
| POST   | /2fa/setup        | Tạo secret + QR code     | JWT        |
| POST   | /2fa/verify       | Xác nhận bật 2FA         | JWT        |
| POST   | /2fa/disable      | Tắt 2FA (cần TOTP code)  | JWT        |
| POST   | /2fa/login-verify | Bước 2 login khi 2FA bật | temp_token |